Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

AI for Network Prediction & IoMT

Focus area: Machine learning models for predicting bandwidth drops and analyzing medical sensor data.

Name: CHARLES (24BIT0522)

Abstract

Problem Statement: The rapid expansion of remote telemedicine and Internet of Medical Things (IoMT) devices generates massive, continuous data streams requiring ultra-low latency processing. Determining where to process this data—locally at the hospital edge or in the centralized cloud—is a critical challenge for modern healthcare infrastructure.

Existing Challenges: Currently, hospitals rely on static cloud architectures that transmit all telemetry data directly to centralized public clouds. This approach faces severe limitations: sudden hospital network congestion can cause life-threatening latency for critical patient alerts. Furthermore, existing systems lack the intelligence to predict network failures, reacting only after data transmission has already failed.

Proposed Solution: This project proposes an AI-Based Serverless Telemedicine Workload Offloader. Specifically, this student component focuses on developing an AI-driven time-series forecasting model to predict hospital network bandwidth drops before they occur. This predictive intelligence empowers the core Multi-Criteria Decision Intelligence (MCDM) engine to proactively route critical medical alerts to local edge processing during network vulnerability, ensuring continuous patient safety while routine analytics are offloaded to the cloud.

AWS Services Used: The architecture leverages AWS IoT Core to securely ingest simulated IoMT telemetry and AWS IoT Greengrass for localized edge execution. Amazon SageMaker is utilized to train, host, and deploy the predictive machine learning model, while Amazon S3 stores the foundational training datasets.

Literature Survey

Paper	Method	Dataset	Advantages	Limitations	Research Gap
1. Deep Learning for Network Congestion Prediction in Healthcare IoT (IEEE, 2025)	LSTM-based time-series forecasting.	5G Network Traces	High accuracy in predicting bandwidth drops under 10ms.	High computational cost to run the model continuously on the cloud.	Need lightweight AI models that can trigger localized edge-processing before the cloud connection fails.

2. Edge-AI in Remote Patient Monitoring Systems (Elsevier, 2024)	Federated Edge Learning (FELL).	MIMIC-III Vitals	Reduces data transmission overhead by 40%.	Does not account for dynamic pricing of cloud failovers during high traffic.	AI must be integrated with cost-aware decision engines, not just latency-aware models.
3.Spatiotemporal Forecasting for Hospital Wi-Fi Reliability (Springer, 2026)	Graph Convolutional Networks (GCN).	Enterprise Wi-Fi logs	Maps physical hospital dead-zones effectively.	Struggles with sudden, unpredictable anomaly spikes	Network prediction must be paired with instant anomaly detection to trigger immediate edge-failover.
4. Energy-Efficient AI Workload Offloading in Wearables (MDPI, 2025)	Deep Reinforcement Learning (DRL).	IoMT Sensor Data	Maximizes battery life of medical wearables.	Black-box decision making; lacks explainability for medical IT audits	Mathematical MCDM models provide better transparency for clinical compliance than pure DRL.
5. Traffic Classification of Telemedicine Video Streams (IEEE, 2023)	CNNs on packet headers	Telehealth Video Packets	Differentiates emergency video feeds from routine calls quickly.	Privacy risks if packet payloads are inspected.	Payload-blind AI models are required to maintain strict HIPAA compliance during classification.

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

Research Gap Analysis

1. Need lightweight AI models that can trigger localized edge-processing before the cloud connection fails.

2. AI must be integrated with cost-aware decision engines, not just latency-aware models.

3. Network prediction must be paired with instant anomaly detection to trigger immediate edge-failover.

4. Mathematical MCDM models provide better transparency for clinical compliance than pure DRL.

5. Payload-blind AI models are required to maintain strict HIPAA compliance during classification.

Project Objectives

1. **ove prediction accuracy using AI** by deploying a time-series forecasting model (via Amazon SageMaker) to proactively predict hospital network bandwidth drops.
2. **Reduce response time using AWS services** by proactively offloading critical telemetry based on AI network forecasts.
3. **Store data securely in the cloud** using Amazon S3 for ML training datasets.
4. **Visualize results using dashboards** by plotting the AI's predicted latency vs. actual

Novelty Summary

What makes this project different from existing work?
Current systems route workloads based on static, reactive rules (waiting for the network to fail). Our solution introduces predictive context-aware routing.

- **New feature:** Integration of predictive network forecasting directly into a real-time routing engine, allowing proactive responses.
- **Better accuracy:** Utilizing dedicated Machine Learning time-series forecasting models (via SageMaker) ensures higher accuracy in predicting true network congestion compared to standard ping-based latency tests.
- **Better AWS integration:** Deep integration of Amazon SageMaker outputs with AWS IoT Core for seamless edge-to-cloud execution.

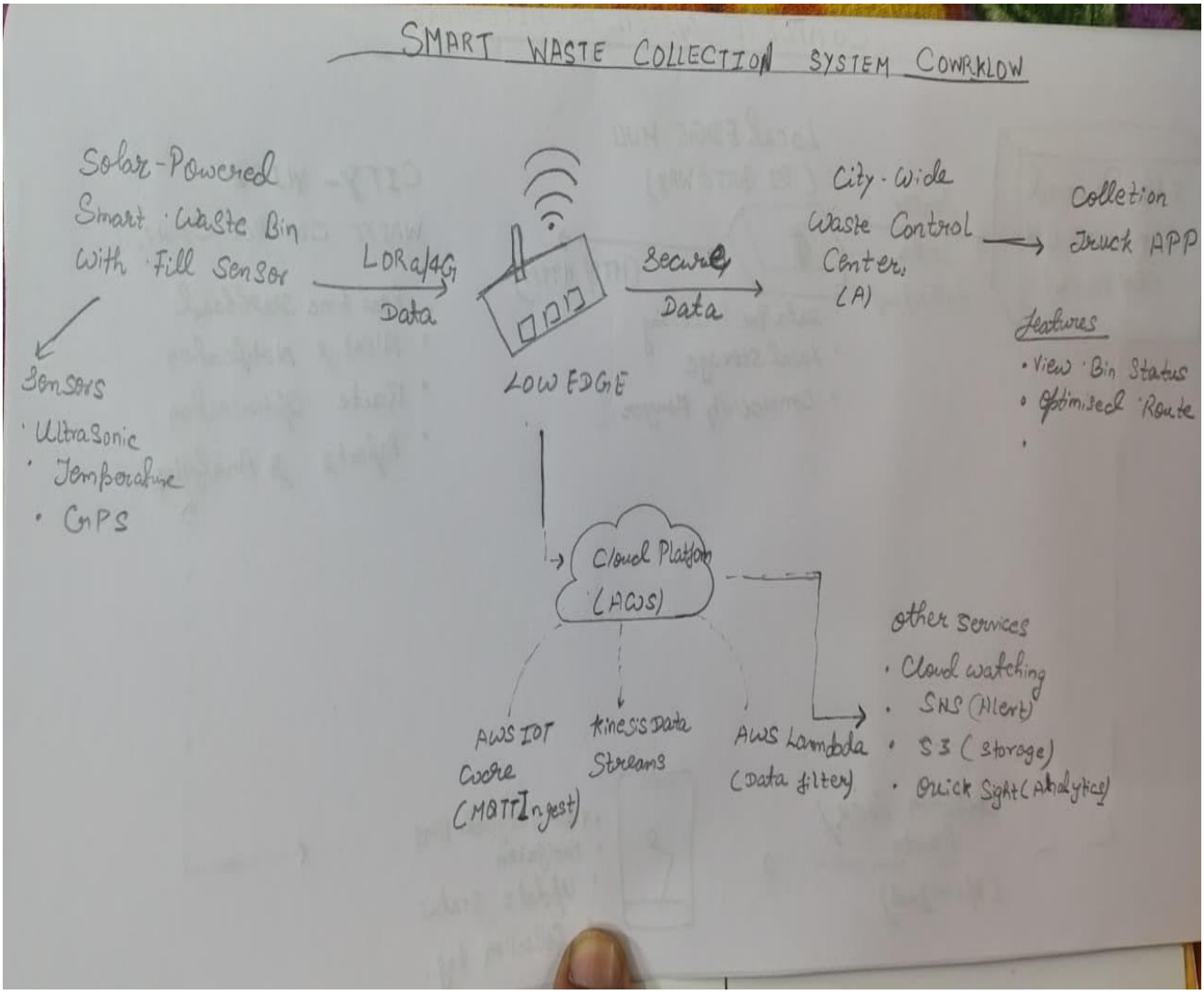
Proposed Architecture

Two architecture diagrams are mandatory.

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

Diagram 1- AWS Cloud Architecture



Course Instructor: Dr. Priya V

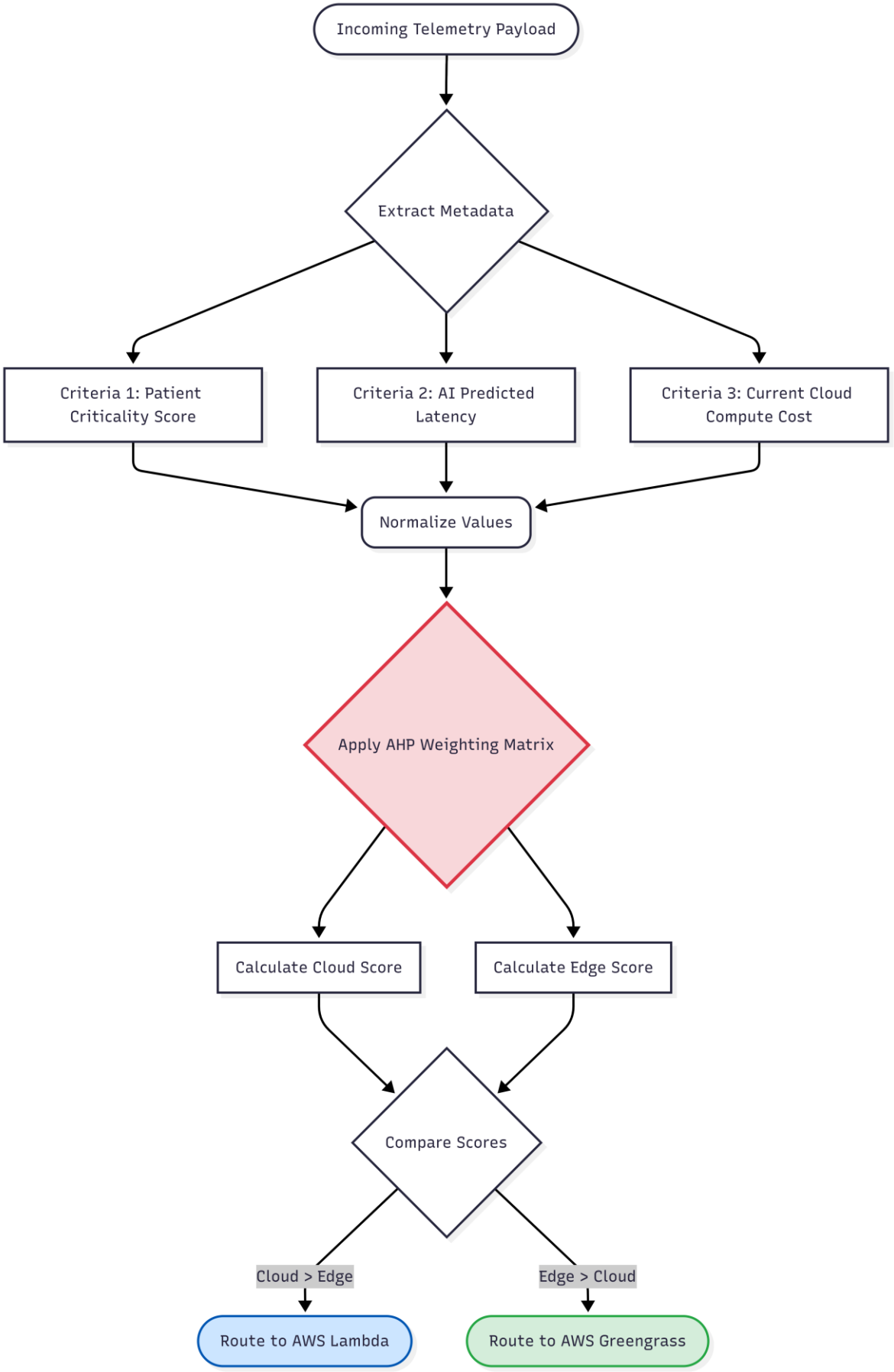
Diagram 2 - Complete System Architecture

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

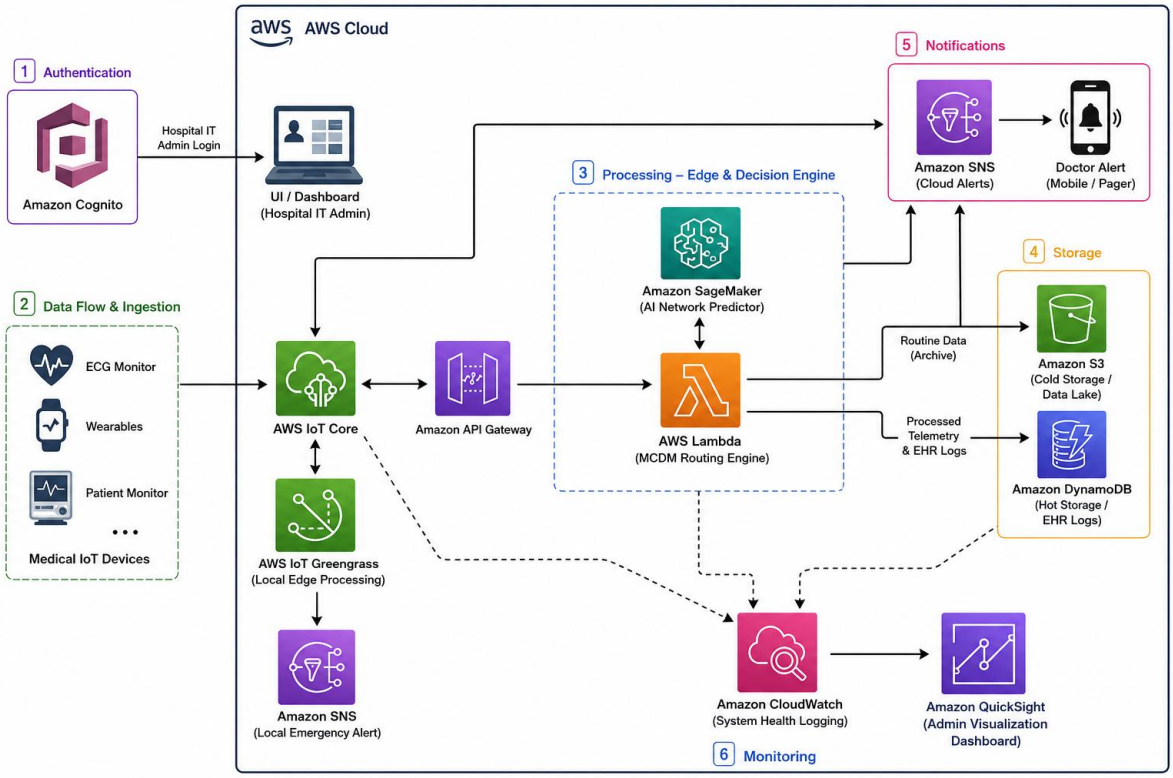
Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V



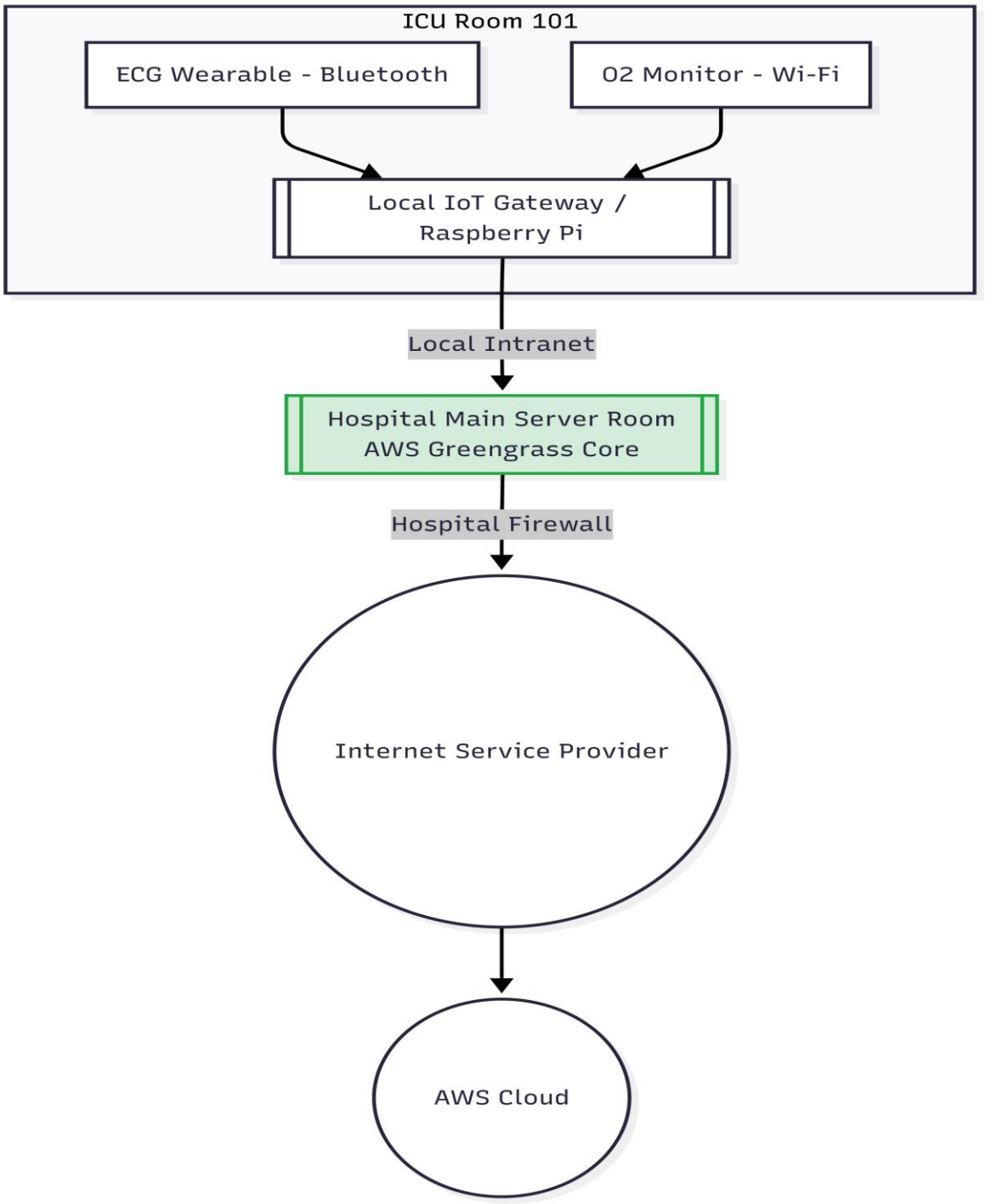
Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V



Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V



Dataset Details

- Source: Synthetically generated locally using Python (NumPy/Pandas).
- Size: Approximately 25 MB.
- Purpose of using the dataset: To train the Amazon SageMaker time-series forecasting model to recognize network degradation patterns.

Course Instructor: Dr. Priya V

- Preprocessing required: Normalization of bandwidth metrics (Min-Max scaling), sliding-window sequencing for LSTM time-series ingestion.

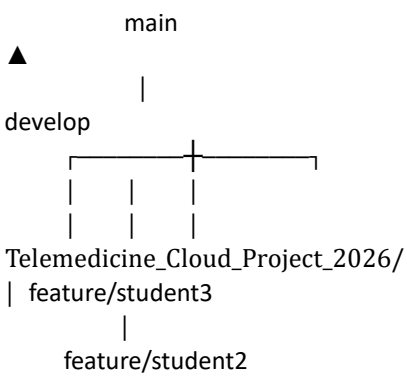
AWS Services Planning

- **Amazon SageMaker:** Train, test, and deploy the AI time-series model to predict network bandwidth drops.
- **AWS IoT Core:** Securely ingest data streams from simulated medical IoT devices.
- **Amazon S3:** Store raw telemetry datasets and ML training data.

GitHub repository

```
Telemedicine_Cloud_Project_2026/
├── dataset/
│   ├── raw/
│   │   └── telemetry_data.csv
│   ├── processed/
│   └── Dataset_Details.docx
├── src/
│   └── ml_model/
│       ├── preprocessing.py
│       ├── train.py
│       ├── predict.py
│       └── model.pkl
├── results/
│   ├── graphs/
│   ├── screenshots/
│   └── accuracy.xlsx
└── docs/
    └── Research_Gap_Student1.docx (Your 5 papers)
```

Final Merge Process



Expected Contribution Matrix - Sample

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

Activity	Student 1	Student 2	Student 3
Literature Survey (5 Papers)	✓	✓	✓
Research Gap (5 Papers)	✓	✓	✓
Frontend Development			
Backend Development			
Database Integration			
Azure Cloud Services			
AI / Machine Learning			
Testing			
Documentation			
Presentation			
GitHub Commits			

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

AI-Based Serverless Telemedicine Workload Offloader

Focus area: MCDM & Backend Logic

Name: Tanmay Kapoor (24BIT0079)

Abstract

The rapid expansion of remote telemedicine and Internet of Medical Things (IoMT) devices generates massive data streams requiring ultra-low latency processing. Determining where to process this data—locally at the hospital edge or in the centralized cloud—is a critical challenge. Currently, hospitals rely on static cloud architectures that transmit all telemetry data directly to public clouds, which incurs massive, unnecessary cloud computing costs for non-critical data.

This project proposes an AI-Based Serverless Telemedicine Workload Offloader that dynamically routes data between local hospital Edge servers and the cloud. Specifically, this component focuses on the development of a Multi-Criteria Decision Intelligence (MCDI) engine. This engine evaluates real-time variables—specifically Patient Criticality, Current Network Latency, and Compute Cost—to mathematically determine the optimal execution environment. By executing an Analytic Hierarchy Process (AHP) algorithm within AWS Lambda, the system autonomously overrides cost-saving cloud routes during emergencies, routing critical data to the Edge and routine data to the cloud, thereby optimizing both patient safety and AWS operational costs.

Literature Survey

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

Paper Title & Publisher	Method	Dataset	Advantages	Limitations	Research Gap	Possible Improvement
1. A framework for offloading and migration of serverless functions in the Edge–Cloud Continuum (Elsevier, 2024)	Serverledge framework with horizontal and vertical function offloading and migration	Experimental evaluation on edge-cloud testbed	Supports live migration and decentralized scheduling with improved utilization	General-purpose framework; not healthcare-aware; no decision intelligence	No medical workload prioritization or telemedicine-specific backend logic	Add AI-based healthcare workload classifier and MCDM-driven migration policy
2. Energy-Aware Tasks Offloading Based on DQN in Medical Mobile Devices(Springer, 2024)	Deep Reinforcement Learning (DQN) for task offloading	Medical mobile device simulation	Reduces execution delay and energy consumption	High computational complexity during training	Limited scalability evaluation for larger networks	Evaluate performance on large-scale healthcare systems and optimize training efficiency

3. Resource Allocation and Data Offloading Strategy for Edge-Computing-Assisted Intelligent Telemedicine System (MDPI, 2023)	Resource allocation and computation offloading optimization	Wireless Body Area Network (WBAN) simulation	Improves transmission efficiency and reduces latency	Security and privacy aspects are minimally addressed	Lack of comprehensive security mechanisms for patient data	Integrate stronger encryption and privacy-preserving techniques
4. DRL-Based Dependent Task Offloading with Delay-Energy Tradeoff in Medical Image Edge Computing (2025)	Deep Reinforcement Learning (DRL).	Medical image processing simulation	Balances delay and energy while handling dependent tasks	Tested mainly on medical imaging workloads	Applicability to broader healthcare services remains unexplored	Evaluate with diverse healthcare applications such as telemedicine, EHRs, and remote monitoring

5. Towards providing a priority-based vital sign offloading in healthcare with serverless computing and a fog-cloud architecture (SmartVSO)(Elsevier, 2024)	Priority-based workload offloading using serverless fog-cloud architecture	Simulated smartwatch vital-sign data under different workloads	Low latency for critical patients, scalable architecture, serverless deployment	Priority rules are static; no AI-based decision making; no cloud provider selection	Lacks intelligent backend capable of dynamically selecting execution location based on multiple factors	Integrate AI + MCDM engine considering latency, cost, server load, patient severity, and reliability simultaneously

Research Gap Analysis

1. Most existing studies are evaluated using simulations or small-scale test environments, with limited deployment in real healthcare settings.

2. Many proposed frameworks are tested under limited workloads and do not assess performance in large-scale or high-demand telemedicine environments.

3. Although healthcare data is highly sensitive, several studies provide limited discussion on data privacy, security, and regulatory compliance during workload offloading.

4. Existing solutions often use fixed or predefined decision rules, making them less effective in handling dynamic changes in network conditions, resource availability, and patient workload.

5. Most research focuses on specific use cases such as vital sign monitoring or medical imaging, while broader telemedicine services (e.g., video consultations, EHR access, remote diagnostics) remain underexplored.

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

Project Objectives

1. **Develop an intelligent cloud-based monitoring system** with a dynamic routing algorithm acting as the core traffic controller.
2. **Reduce response time using AWS services** by mathematically prioritizing latency over cost for critical patient alerts.
3. **Store data securely in the cloud** by routing routine, processed logs to Amazon DynamoDB for hot storage.
4. **Visualize results using dashboards** by utilizing Amazon QuickSight to display the real-time routing decisions and cost-savings achieved by the MCDM engine.
5. **Improve routing automation** by coding strict override protocols within the decision matrix for clinical emergencies

Novelty Summary

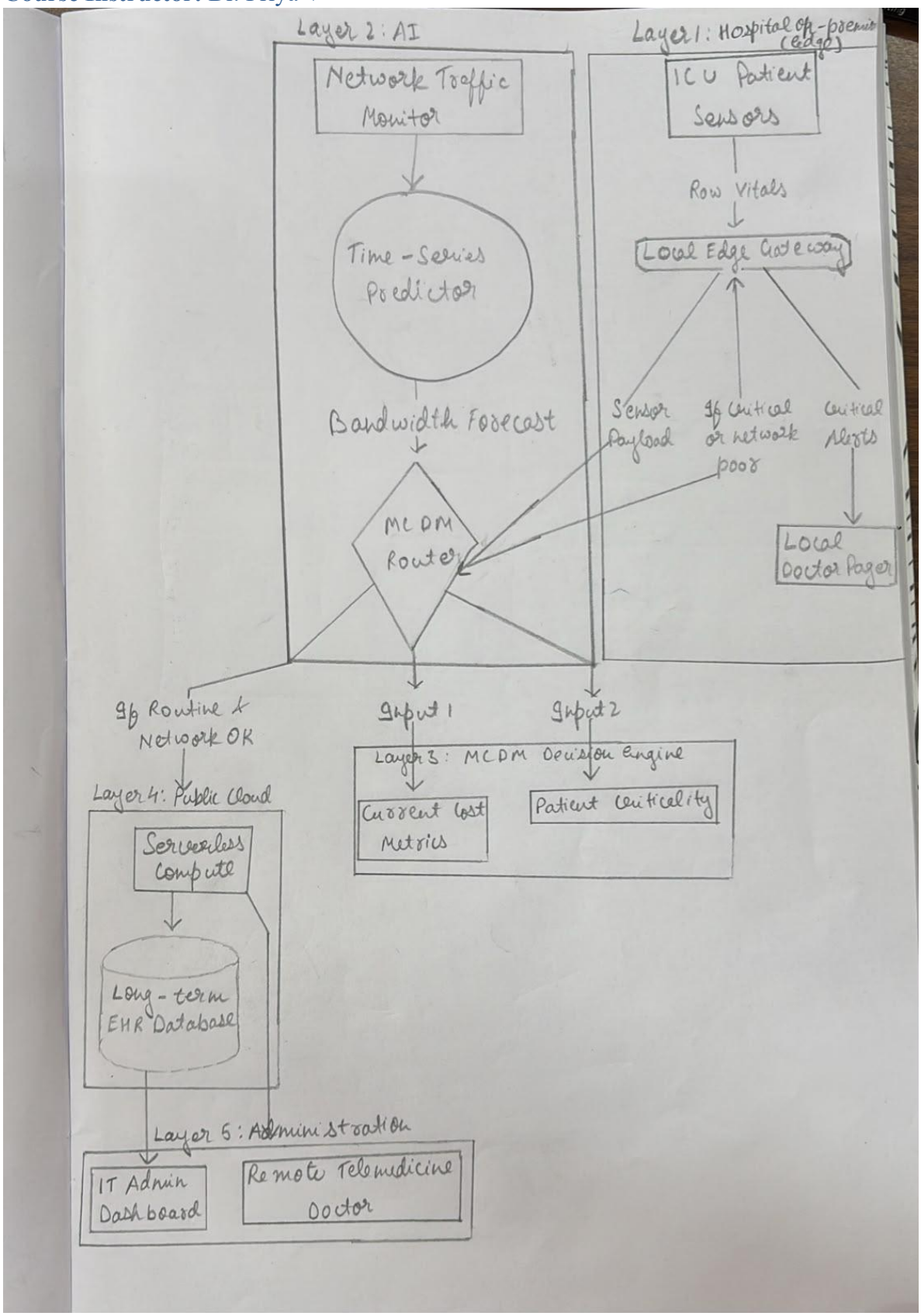
What makes this project different from existing work?
Existing cloud migration systems rely on simple binary threshold checks (e.g., If Latency > 100ms, then switch). Our solution implements complex mathematical balancing.

- **Better algorithm:** Implements a Multi-Criteria Decision Intelligence (MCDM / AHP) algorithm that dynamically recalculates the balance between Patient Criticality, Latency, and AWS Cost in real-time.
- **Better automation:** Features autonomous override routing that automatically locks execution to the Edge during "Code Blue" clinical emergencies, requiring no human IT intervention.
- **Better scalability:** Achieves infinite scaling by executing the complex decision math entirely within event-driven AWS Lambda functions.

Proposed Architecture

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V



Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V

Dataset Details

- **Dataset Name:** Telemedicine IoMT Network and Criticality Telemetry (Synthetic 2026)
- **Source:** Synthetically generated locally using Python (NumPy/Pandas).
- **Purpose of using the dataset:** To serve as a continuous simulation input stream to test the real-time routing capabilities and mathematical accuracy of the MCDM Lambda engine.

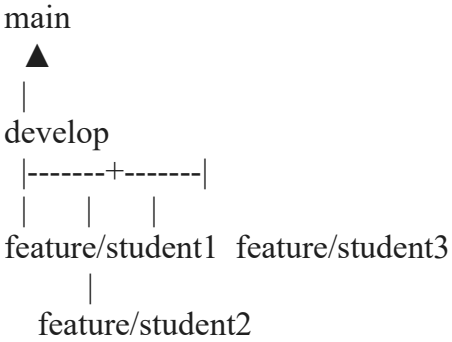
AWS Services Planning

- **AWS Lambda:** Serve as the core serverless compute engine to execute the MCDM (AHP) mathematical routing logic.
- **Amazon API Gateway:** Provide REST APIs to securely route inbound telemetry to the decision engine.
- **Amazon DynamoDB:** Store processed patient telemetry logs and the history of routing decisions for rapid, hot access.
- **Amazon QuickSight:** Visualize latency savings and routing statistics on an interactive admin dashboard.

GitHub Guidelines

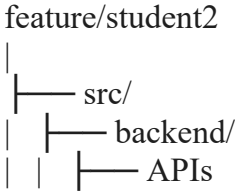
Assigned Git Branch: feature/student2

GitHub Branch Allocation Workflow:



Student B Folder Structure & Responsibilities:

Student B is strictly responsible for managing and pushing code to the following folders in the shared repository:



Course Instructor: Dr. Priya V

```
|   |   ├── Controllers
|   |   ├── Database
|   |   └── Authentication
|   |
|   └── aws/
|       ├── Lambda
|       ├── S3
|       ├── EC2
|       ├── IAM
|       └── CloudWatch
|
├── docs/
|   └── Research_Gap_Student2.docx
|
└── architecture/
    └── AWS_Architecture.png
```

Final Merge Process:

- Commit work regularly to feature/student2.
- Raise Pull Requests (PRs) from feature/student2 to develop.
- Do not commit directly to main or develop.

Expected Contribution Matrix - Sample

Activity	**Student 1**	**Student 2**	**Student 3**
Literature Survey (5 Papers)	✓	✓	✓
Research Gap (5 Papers)	✓	✓	✓

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026

Serverless Architectures & Healthcare Security
Focus area: AWS infrastructure (Lambda, Greengrass) and HIPAA compliance.

Name: Rani Gupta (24BIT0543)

Abstract

Problem statement: The rapid expansion of remote telemedicine and Internet of Medical Things (IoMT) devices generates massive, continuous data streams requiring ultra-low latency processing. Determining where to process this data—locally at the hospital edge or in the centralized cloud—is a critical challenge for modern healthcare infrastructure.

Existing challenges: Existing systems often transmit all highly sensitive patient data (Protected Health Information or PHI) continuously over public networks. This creates significant HIPAA compliance risks, especially during periods of network vulnerability or cyberattacks. Additionally, traditional monolithic server architectures fail to scale efficiently to handle sudden influxes of thousands of concurrent medical sensor streams.

Proposed solution: This project proposes an AI-Based Serverless Telemedicine Workload Offloader. Specifically, this student component focuses on designing the robust, serverless hybrid-cloud architecture required to execute dynamic routing securely. The topology ensures that critical alerts are processed immediately on local hospital hardware, preventing sensitive PHI from traversing the public internet during congestion. The design enforces strict event-driven scalability and integrates comprehensive access control to guarantee adherence to healthcare data protection standards.

AWS services used: The hybrid architecture bridges on-premise hardware using AWS IoT Greengrass with the public cloud via AWS Lambda. AWS IAM enforces strict role-based access control for compliance, while Amazon Cognito authenticates administrative users. Amazon CloudWatch provides essential system observability and error logging, ensuring the overarching Edge-to-Cloud framework remains highly available and secure.

Literature Survey

Paper	Method	Dataset	Advantages	Limitations	Research Gap
11. Performance Analysis of Edge	Empirical latency testing.	Local Sensor to Edge Core	Proves Edge processing is 10x faster than cloud for local networks.	Does not address how to sync Edge data back to Cloud seamlessly.	Need a hybrid synchronization mechanism to ensure Edge data eventually reaches long-term Cloud EHRs (S3/DynamoDB).

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026

12. Cold Start Mitigation in Serverless Functions for Healthcare APIs (ACM, 2026)	Concurrency tuning.	Serverless API Traces	Reduces Lambda cold starts to under 50ms.	Expensive to maintain provisioned concurrency 24/7.	Predictive AI should be used to scale AWS Lambda concurrency *only* when network traffic dictates.
13. HIPAA-Compliant Architectures in Multi-Cloud Environments (Springer, 2024)	Framework design & mapping	Regulatory Rulebooks	Comprehensive security guidelines for PHI	Abstract; lacks automated enforcement tools.	Compliance rules must be codified directly into the MCDM routing logic as strict mathematical constraints.
14. Blockchain vs. Serverless for EHR Access Logs (Elsevier, 2025)	Comparative architecture review.	Simulated EHR queries	High integrity for audit trails.	Blockchain is too slow for real-time IoMT telemetry	High-speed serverless logging combined with secure object storage is a better alternative for fast IoT telemetry.
15. Scalability of Microservices in Telehealth Platforms (Wiley, 2026)	Kubernetes (EKS) stress testing.	Video consultation data	Highly scalable for heavy video processing.	Overkill and complex for lightweight sensor telemetry	Event-driven Serverless (AWS Lambda/Greengrass) is more efficient for IoT telemetry than heavy container orchestration.

Research Gap Analysis

1. Need a hybrid synchronization mechanism to ensure Edge data eventually reaches long-term Cloud EHRs (S3/DynamoDB).

2. Predictive AI should be used to scale AWS Lambda concurrency only when network traffic dictates.

3. Compliance rules must be codified directly into the MCDM routing logic as strict mathematical constraints.

4. High-speed serverless logging combined with secure object storage is a better alternative for fast IoT telemetry.

5. Event-driven Serverless (AWS Lambda/Greengrass) is more efficient for IoT telemetry than heavy container orchestration.

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026

Project Objectives

1. **Develop an intelligent cloud-based monitoring system** by constructing a secure Hybrid-Cloud topology spanning hospital premises and AWS.
2. **Reduce response time using AWS services** by properly configuring AWS IoT Greengrass to execute localized processing.
3. **Store data securely in the cloud** using Amazon S3 and Amazon DynamoDB, ensuring all Protected Health Information (PHI) is encrypted and strictly HIPAA-compliant.
4. **Visualize results using dashboards** by configuring secure admin access via Amazon Cognito.
5. **Ensure high system observability** by implementing comprehensive system health logging using Amazon CloudWatch.

Novelty Summary

What makes this project different from existing work?

Current architectures rely on monolithic, centralized cloud servers where all data must travel to a remote data center. Our project introduces a seamless Edge-to-Cloud topology.

- **Better architecture:** Utilizes a Serverless Edge with public cloud infrastructure (AWS Lambda).
- -to-Cloud Hybrid architecture, seamlessly bridging local hospital hardware (AWS Greengrass)**Better security:** Enhances security by processing highly sensitive Protected Health Information (PHI) locally at the Edge during periods of network vulnerability, significantly reducing exposure risk.
- **Better AWS integration:** Achieves deep, event-driven integration across AWS IoT Core, Greengrass, API Gateway, and CloudWatch without relying on legacy Virtual Machines (EC2).

Proposed Architecture

Diagram 1- AWS Cloud Architecture

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026

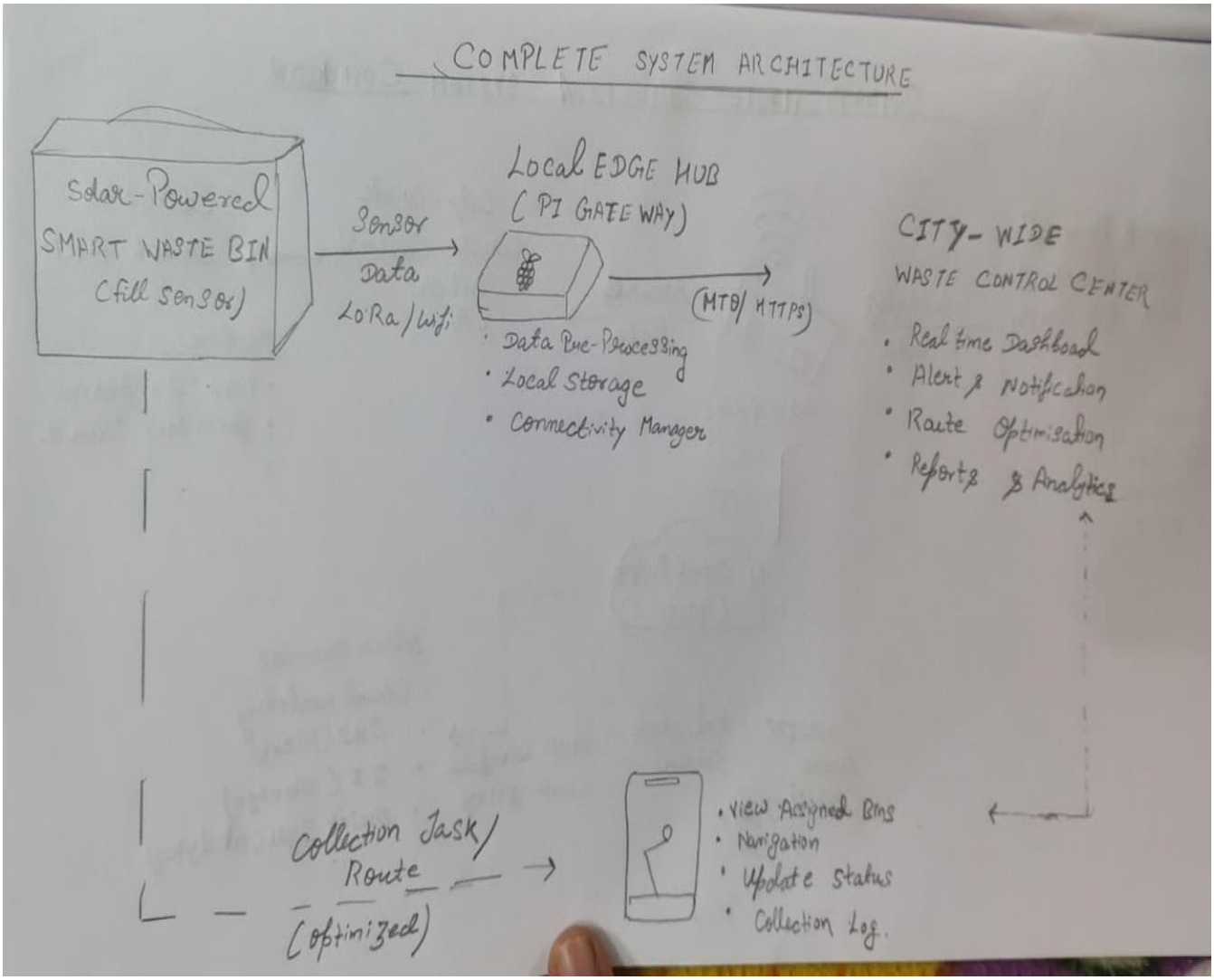
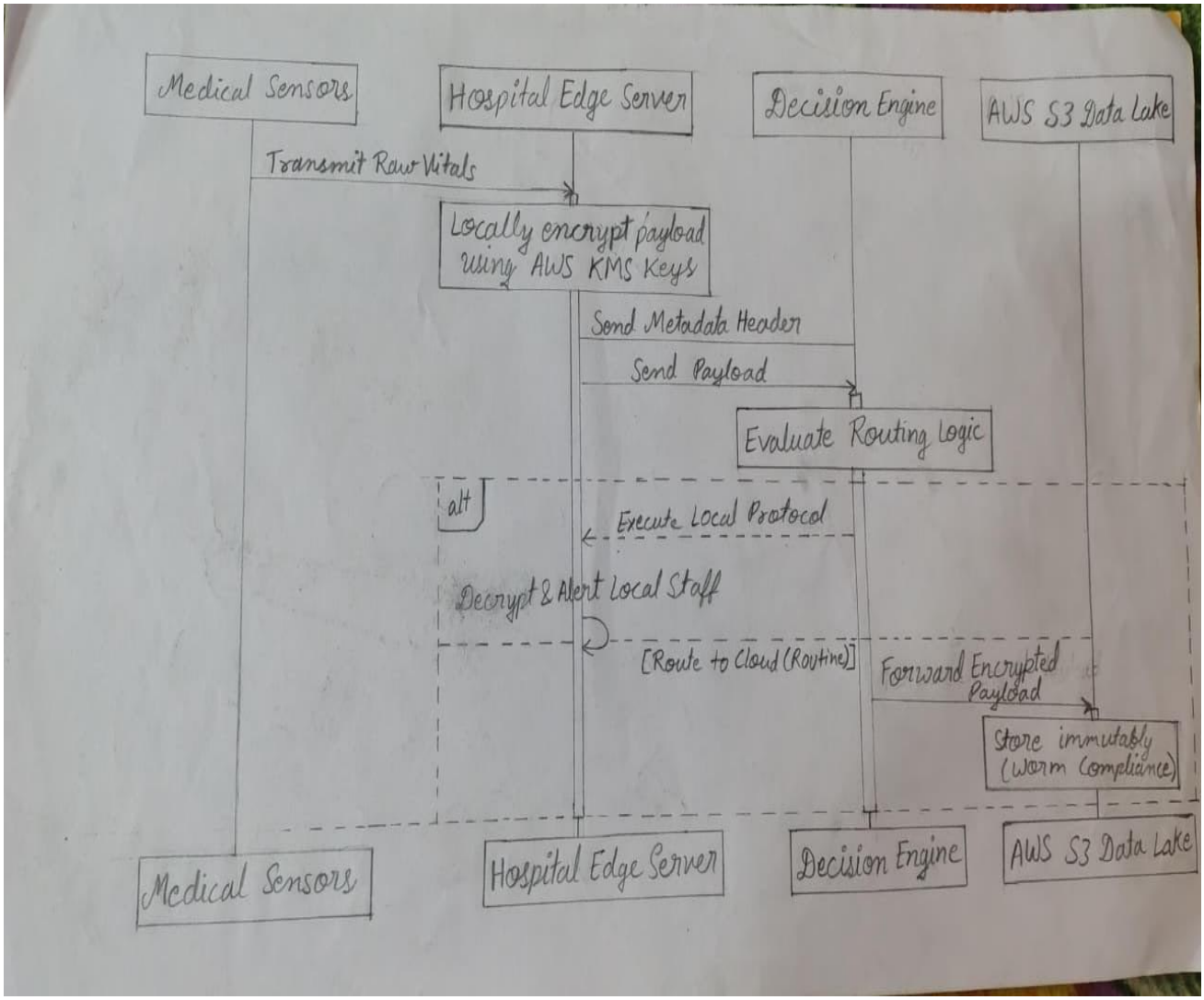


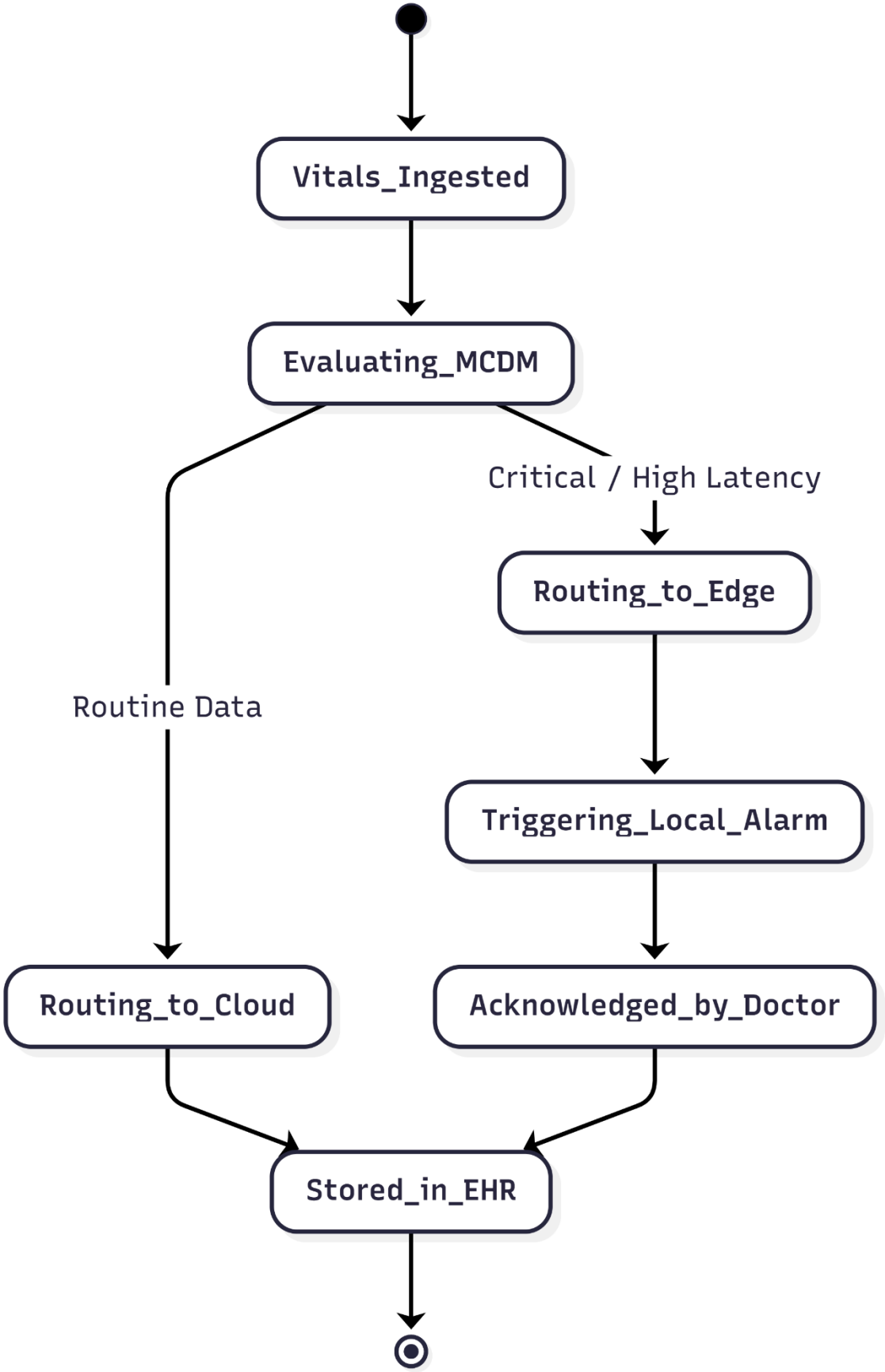
Diagram 2 - Complete System Architecture

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026



Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026



Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026

Dataset Details

Dataset Name: Telemedicine IoMT Network and Criticality Telemetry (Synthetic 2026)

Purpose of using the dataset (Infrastructure view): To simulate high-throughput JSON payload ingestion into AWS IoT Core to test the elasticity and latency of the serverless architecture.

AWS Services Planning

- **AWS IoT Greengrass:** Execute localized processing and trigger alerts at the hospital edge when cloud latency is critical.
- **Amazon Cognito:** Authenticate hospital IT admins securely before accessing the dashboard.
- **AWS IAM:** Enforce strict role-based access control (RBAC) to ensure HIPAA compliance across all services.
- **Amazon CloudWatch:** Monitor overall system health, track Lambda execution latencies, and log routing errors.

GitHub Guidelines

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

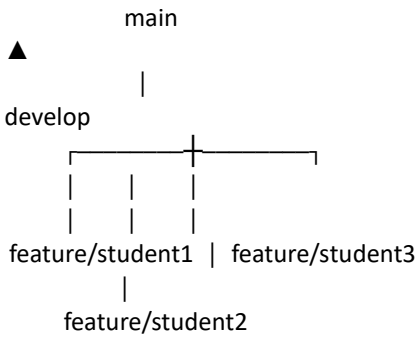
Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026

feature/student3

```
├── dataset/
│   ├── raw/
│   ├── processed/
│   └── Dataset_Details.docx
│
├── src/
│   └── ml_model/
│       ├── preprocessing.py
│       ├── train.py
│       ├── predict.py
│       └── model.pkl
│
├── results/
│   ├── graphs/
│   ├── screenshots/
│   └── accuracy.xlsx
│
└── docs/
    └── Research_Gap_Student3.docx
```

GitHub Branch Allocation

Final Merge Process



Workflow

- Create the repository.
- Create the develop branch.
- Each student creates their own feature/studentX branch.

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026

- Students commit their work regularly to their branch.
- Raise a Pull Request (PR) from the feature branch to develop.
- Team members review the changes.
- Resolve any merge conflicts.
- Merge approved changes into develop.
- Once the project is stable, merge develop into main.
- Tag the release (e.g., v1.0-Phase1).

Expected Contribution Matrix - Sample

Activity	Student 1	Student 2	Student 3
Literature Survey (5 Papers)	✓	✓	✓
Research Gap (5 Papers)	✓	✓	✓
Frontend Development			
Backend Development			
Database Integration			
Azure Cloud Services			
AI / Machine Learning			
Testing			
Documentation			
Presentation			
GitHub Commits			

Project Phase-I Guidelines - BCSE355L - Cloud Architecture Design

Course Instructor: Dr. Priya V
Submission Deadline: 30 July 2026